

Specification of a diagrammatic reasoning tool (v2)

Ambroise LAFONT

April 20, 2026

This is a description of a tool that allows to reason graphically. It does not supersede the previous version. The focus, here, is on more technical details regarding the implementation of the graphical interface. We don't cover the interfacing with Rocq.

We envision a web application where the user can specify a schema (i.e., the involved algebraic structures and how they are drawn) as well as rewriting rules. The actual technology should provide a way to load js script lively without need to recompile the full web app. This is not a big requirement since web apps are typically based on js which have this feature built-in.

A big technical challenge is to design the format of specification of drawing of algebraic structures. Our proposed solution is (at least in the first versions) to take as input a js script, relying on some infrastructure that we provide.

Of course, we do not expect any user to write js code, so we will have two kinds of users.

- The *modder* will be able to write js code directly, and thus specify schemas and implement some facilities for other modders;
- The *end user* will only interact with a graphical interface: they will be able to choose from a list of pre-defined schemas, and then use the graphical interface to specify rewriting rules, and prove things using those rules.

1 Vocabulary

Every list of sort declarations defines what we call a *schema*. For instance, the schema for categories has four sorts: *ob* and *hom*, where *hom* doubly depends on *ob*, and *id* and *comp*, which suitably depend on *hom*.

We call a *complex* a finite model of a schema. For instance, for the schema of categories, a complex consists of a finite set of objects, a finite set of morphisms between those objects, and some of those are marked as identities, and some triangles of morphisms are marked as composition triangles. Note that such a complex may not be a category. In particular, there is a complex consisting of

a single object and no morphism. We call an object, morphism, or whatever, a *semantic element* of the complex.

A *diagram* is a drawing of a complex, in the software. Two different diagrams may represent the same complex. For instance, in the schema of categories, two diagrams that differ by the position of an object, or by the bending of an arrow, should be considered as representing the same complex. But also, a diagram may include multiple representations of the same semantic element. More specifically, we call a *diagram element* a visual representation of a semantic element. Typically we do not represent an identity morphism (which is actually an endomorphism) as a loop, but as an arrow between two different representations of the same object. In that case, we have two diagram elements (the diagram objects) representing the same semantic object.

We take this a principle: the drawing of an element should always occur in a context where all the diagram dependencies are distinct (unless the *modder* has explicitly provided additional support for non-distinct dependencies, such as loops, in the case of arrows, but let's keep this idea for future work).

We say that two *diagram elements* are *equivalent* if they represent the same *semantic element*.

2 Rules

A rule is a valid transformation of the drawing with respect to the semantic models (e.g., categories). A proof trace consists of a sequence of rules. There are three kinds of rules:

- *Semantic rules* are structures supported by the models (e.g., for every pair of composable arrow, there is a composite arrow);
- *Data rules* are rules that modify the data of the *diagram elements*, and thus aspects of the drawing (e.g, the positions of the objects, the bending of the arrows, the labels, etc.) that are not relevant for the semantics;
- *Equality rules* are rules that modify the drawing by duplicating a semantic element, or merging two semantic elements with the same labels.

When storing the proof trace, we remember the sequence of rules. Adjacent data rules should be merged, and same for equality rules.

We call a rule *backward* if it introduces subgoals (e.g., to prove that a morphism is mono, we must show that for every pair of parallel morphisms, if they are equal after post-composing with the morphism, then they are equal), and *forward* otherwise (e.g., given two composable morphisms, we can compose them). The only potential source of backward rules are the semantic rules.

The definition of a backward rule may specify labels for the subgoals.

2.1 Equality rules

Any pair of equivalent *diagram elements* can be merged, as long as it is still possible to draw any element that depends on those. For example, if the *modder*

does not provide support for drawing loops, then it is not possible to merge equivalent two objects if there is a morphism between them. When two elements are merged, the labels are concatenated (remember that labels are lists of strings), eliminating redundancies.

On the other hand, it is always possible to duplicate a [diagram element](#).

3 Interface for proving

Here we list the panels that we envision for the interface.

- The workspace displaying the current state as a diagram. Depending on what the mouse is focusing on (or even just hovering in certain cases) in the other panels, some elements could be highlighted (for example, the other diagram elements could be made opacity 50 percent). As an example, if rule is selected, the match of the rule should be displayed.
- Tree of goals, with two kinds of internal nodes: those labelled by the name of the [backward](#) rule, and those labelling the subgoals (remember that a backward rule includes labels for the subgoals, e.g., uniqueness and existence for a pullback). One of node/leaf should be selected at any time. A node marked with a checkmark is a node whose proof is complete. When it is focused (i.e., when nothing in the proof trace is selected), it is displayed as a 2-layered diagram in the workspace view (cf next section for the notion of layers), i.e., as two tabs.
- Proof trace, represented as a list of rules¹. Each line is labelled like "before name-of-the-rule", to mean that this is the state before applying the rule. [Data rules](#) or [Equality rules](#) are named "before 'data'", or "before 'equality'".

Contextual menu:

- animate by interpolation (for data and equality rules) the change until the next state;
- highlight elements modified by previous rule
- highlight elements modified by next rule (if this is not the last state)
- restart the proof from this state.
- Forest of diagram elements, sorted by their sorts. The children of an element are its dependencies, with the possibility to search for labels, and selecting elements. Elements that have been modified/created by the previous rules should have a specific highlight, and similarly for those modified by the next rule.

Contextual menu:

¹Groove rather displays a graph, which is also interesting because it allows for backtracking and saving efforts

- hide (could apply to all elements of some sort, e.g., you may want to hide all composition witnesses?)
- merge with (and then you have to select another equivalent element, where the panel could restrict to actual possibilities, and hovering would highlight them in the workspace)
- duplicate (and then you have to select dependencies one by one, that must be equivalent to the other ones. The actual possibilities could be highlighted both in the forest and on the workspace)
- The data panel associated with the current selected diagram element, with the possibility to edit the data (e.g., position, color, etc.). For position-like data, this is done by the mouse.
- The equality panel, listing all the non-trivial equivalence classes as a list of (distinct) labels.
- 2-depth tree of rules that can be applied in the current context (like in Groove). Roots are the names of the rules, and the children are leaves with labels where it applies. Rules that would not change the context (e.g., because they merge two elements that are already merged) or add stuff that is already there (e.g., composition of two morphisms when they already exist) should not be displayed. It would be nice also to detect that a rule is symmetric to avoid duplicate occurrences (e.g., the rule that equalises two parallel morphisms when postcomposed by a mono is symmetric).

Contextual menu:

- apply rule. Whenever we apply a rule, the software always automatically tests whether the goal has been reached, in that case the current goal is marked as done.

4 Interface for drawing

A drawing can involve up to three layers. Each layer is a chunk of the diagram as well as the involved equivalence classes for diagram elements. A drawing with one layer is typically the state of the current goal when we are in the view proof. The second layer is typically used to specify the output of a **forward** rule, or the statement that we want to prove, given data specified in the first layer. The third layer is used in the specification of the premise of a backward rule. Indeed, consider, as an example, the case of monomorphism. The first layer consists of one morphism, the second layer adds two parallel morphisms such that their composite are equal, and the third layer adds an equality between those two parallel morphisms. This is the premise of the **backward** rule allowing to tag a morphism as mono.

When we create a new **forward** rule, two tabs are created, one for each layer. When we create a new **backward** rule, two tabs are created: one is for the first

layer common to all the other layers, and the other one is for the conclusion. On top of that we have 2 tabs (2 layers) per premise. Consider the example of pullback: the first tab is just a commutative square; the second tab is the same but with the pullback sign. We have two premises: one for existence of a mediating arrow (with one layer with the cone, and the second layer including the mediating arrow), and the other one for uniqueness (one layer has a cone and two mediating arrow, and the other one has an additional equality between the two mediating arrows).

When editing a tab/layer, we cannot edit any previous layer. We have the equality panel and the elements panel. In the elements panel, contextual menu: mark it as equal with for elements of the previous layers, merge with for new elements.

There is an additional panel that allows to select a sort for which we want to add an element.

It would be nice to let the user provide some js script to specify how the data of the new elements should be initialised (e.g., for composition, we may want to have $f;g$ in the label.). When creating a new element that has a label in common with some other element, then an equality is automatically added.

The contextual menu for equalities now includes "remove".

We may consider a color scheme to visualise the different layers on the same drawing. The reason I do not emphasize too much is because it forbids using colors when drawing elements.

5 Specification of sorts by modders

The [modder](#) should provide some default data when creating a new dependent data. Below is the data associated with a sort:

- a name;
- its named dependencies;
- whether it is [propositional](#): that means that there is at most one element of that sort given some dependencies (e.g., mono tag, or pullback sign, or composition triangle);
- a list of data fields with their types (string/int/float/position), this is used when one wants to modify their values with a [data rule](#). A position can be edited by the user by the mouse. There is always an implicit label field which is a list of strings.

The provided script that draws (or initialises as above) should have access to both children and parents. For example, drawing an arrow depends on whether it is mono or epi. Drawing a pullback sign requires information about the children arrows, to draw it at an appropriate location.

Here is an example of style of script that we envision.

```

var Ob = new Sort({}, { pos : {x: int, y :int} });
var Hom = new Sort({source : Ob, target:Ob}, {bend:float});

Ob.drawSVG = function() {
    svg.drawCircleWithLabel(this.pos, this.label);
}

Hom.drawSVG() = function(){
    svg.drawBentLineWithLabel(this.source.pos,
        this.target.pos, this.bend, this.label);
}

```

We should decide on a js drawing library. Of course raw svg could work as a start, but it put more work on modders (e.g., svg provides no facility to position the labels of the arrows).

6 Main menu

- Edit schema
- Prove a lemma: provide a name for the lemma and a statement in the shape of a 2-layered diagram.
- Edit list of rules: all the rules are listed
 - edit rule.
 - new forward rule.
 - new backward rule.
 - remove rule.